KIPI: Project Structure

Status: planned. Nothing in this tree has been written or verified. Line counts are omitted because no code exists yet. Status values: `TODO` (not started), `WIP` (in progress), `DONE` (written and tested), `VERIFIED` (a team member can explain it and it matches the spec).

Repository: `kipi`. A single-file key-value database that survives `kill -9`, its torture tool, and the required documents. Go, standard library only, no dependencies.

Build and run (target)

```
go build -o . ./cmd/kvdb ./cmd/torture
./torture --iterations 500 --seed 12345
go test ./...
```

On Windows the binaries are `kvdb.exe` and `torture.exe`, run as `.\kvdb.exe` and `.\torture.exe`.

Folder tree

```
kipi/
├── go.mod                    module kipi, Go 1.21+                          TODO
├── .gitignore                binaries, torture-work/, *.kv, *.lock, *.tmp TODO
├── README.md                 build, commands, serve protocol, on-disk     TODO
│                             format with hex example and diagram,
│                             recovery in plain English, serve lanes,
│                             sabotage build disclosure (if used)
├── ARCHITECTURE.md           diagram, 32-row proof table, three           TODO
│                             guarantees, open-items results
├── BREAKING.md               crash points CP1–CP14, ordering argument,    TODO
│                             weaknesses, failed attempts, next 24 hours
├── PROMPTS.md                AI prompt log, unedited, in order            TODO
├── DESIGN-EXTENSION.md       written answer to the expiry question        TODO
├── diagrams/
│   ├── architecture.mmd      diagram source (Mermaid)                     TODO
│   └── architecture.png      rendered diagram                             TODO
├── src/kvdb/                 ENGINE (library)
│   ├── format.go             header, record and marker encoding, CRC32C   TODO
│   ├── db.go                 DB type, options, errors, Get, Close         TODO
│   ├── write.go              Apply steps 1–12, disk-full guard            TODO
│   ├── recovery.go           Open(): lock, tmp cleanup, scan, truncate    TODO
│   ├── compact.go            compaction: tmp, fsync, rename, index        TODO
│   ├── check.go              read-only integrity check                    TODO
│   ├── platform_posix.go     flock, dir fsync, F_FULLFSYNC on macOS       TODO
│   ├── platform_windows.go   LockFileEx, close-rename-reopen              TODO
│   └── kvdb_test.go          unit tests                                   TODO
├── cmd/kvdb/                 command-line wrapper → ./kvdb
│   ├── main.go               open put get del batch dump check close      TODO
│   └── serve.go              stdin protocol, lanes, ACK under write lock  TODO
├── cmd/torture/              torture tool → ./torture
│   ├── main.go               flags, iteration loop, coverage, failure     TODO
│   │                         bundle
```

```
│   ├── session.go          serve child, writers, SIGKILL, open/check/   TODO
│   │                       dump
│   ├── oracle.go           independent oracle and verify()              TODO
│   ├── gen.go              modes: default, large-values, compaction,    TODO
│   │                       recovery-crash, full-disk
│   └── scale.go            1M-key recovery time and shrink proof        TODO
└── tests/
    └── e2e_test.go         builds binaries, kills serve, smoke-runs     TODO
                            each mode
```

Optional: `sabotage_off.go` and `sabotage_on.go` in `src/kvdb/`, which add four deliberate bugs under `-tags sabotage` to show the torture tool can fail. Add them only if the team wants that proof, and disclose them in README if they exist.

What each file must do

Engine: `src/kvdb/`

File	Responsibility	Look for	Status
`format.go`	16-byte header `KIPI`, 12-byte record header, commit marker, CRC32C, length validation before reading payloads	`encodeHeader`, `appendRecord`, `encodeMarker`, `payloadLen`, `verifyPut`	TODO
`db.go`	`DB` struct (index, fileSize, liveBytes, RWMutex, poisoned), options, error codes, `Get`, sorted iteration, `Close`	`Get`, `ForEachSorted`, `Close`	TODO
`write.go`	Single write path: validate, disk-full guard, append records, barrier 1, marker, barrier 2, index update, ACK, auto-compaction, rollback on error	`Apply` (steps 1–12), CP1–CP7, `spaceGuard`, `rollback`	TODO
`recovery.go`	`Open()`: take lock first, create or clean tmp, scan the log, truncate the torn tail, ready	`Open` (phases 0–3), `scan`, `createFile`, CP8–CP10, CP14	TODO
`compact.go`	Write live records to `db.kv.tmp`, fsync, rename over `db.kv`, point index at new offsets, fsync directory	`compactLocked` (steps 1–8), CP11–CP13	TODO
`check.go`	Read-only verification for `./kvdb check`. Reports an uncommitted tail without changing the file	`Check`	TODO
`platform_posix.go`	Linux and macOS: `flock`, directory fsync, `F_FULLFSYNC` on macOS, rename where the tmp	`lockFile`, `syncDir`, `installCompacted`	TODO

File	Responsibility	Look for	Status
	descriptor becomes the database descriptor		
`platform_windows.go`	Windows: `LockFileEx`, no directory fsync, close handles, rename, reopen	`lockFile`, `installCompacted`	TODO
`kvdb_test.go`	Round trip, torn tail at every byte offset, uncommitted batch, F4, ordering, compaction, disk-full, lock, concurrency, readers	`go test ./src/...`	TODO

Command line: `cmd/kvdb/`

File	Responsibility	Look for	Status
`main.go`	One-shot commands (`open`, `put`, `get`, `del`, `batch`, `dump`, `check`, `close`) and shared flags (`--max-bytes`, `--hex`, `--trace`, `--compact-min-bytes`, `--step-delay-us`)	`run`, `parseArgs`, `readBatchFile`	TODO
`serve.go`	Long-running mode. Text and hex dialects, `BATCH n`, lanes by op-id prefix, ACK printed under the write lock	`serve`, `parseRequest`, `handle`	TODO

Torture tool: `cmd/torture/`

File	Responsibility	Look for	Status
`main.go`	Flags (`--iterations`, `--seed`, `--mode`), iteration loop, recovery-crash kills, per-iteration line, coverage report, failure bundle	`loop`, `account`, `fail`, `preload`	TODO
`session.go`	Runs `serve` as a child, 2+ concurrent writers, random-millisecond `Process.Kill`, ACK reader, open/check/dump helpers	`runServe`, `handleLine`, `killOpen`, `runDump`	TODO
`oracle.go`	Independent model of ACKed operations. Dump must equal all ACKed ops, plus at most one whole in-flight op	`model.apply`, `verify`	TODO
`gen.go`	Workload profile per mode, key space, op generator, self-describing values	`profileFor`, `gen`, `makeValue`	TODO
`scale.go`	1,000,000 keys: kill mid-batch, time recovery (under 2 s), verify every value, delete all, compact, confirm the file shrinks	`runScale`	TODO

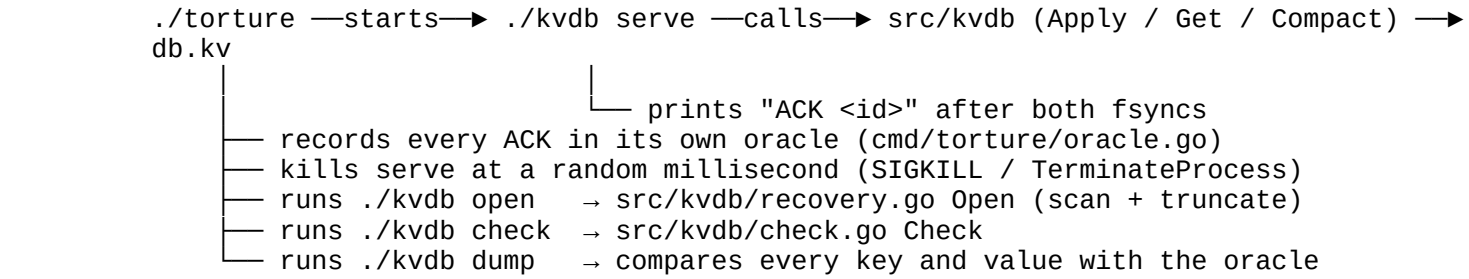
Tests

File	Responsibility	Run with	Status
`src/kvdb/kvdb_test.go`	Unit tests for the engine (see table above)	`go test ./src/...`	TODO
`tests/e2e_test.go`	Builds real binaries, kills `serve` by hand several times, smoke-runs every torture mode	`go test ./tests/`	TODO

Documents

File	Contents	Judge use	Status
`README.md`	Build command, every command, serve protocol, on-disk format with hex example and diagram, recovery in plain English, torture usage	Minute 0–1	TODO
`ARCHITECTURE.md`	Diagram, all 32 scenario rows with code location and the run that verified each, three guarantees	Spot-checks 3 rows	TODO
`BREAKING.md`	CP1–CP14 crash-point table, ordering argument, weaknesses, failed attempts, next 24 hours	Crash-here questions	TODO
`PROMPTS.md`	Chronological AI prompt log: tool, approximate time, prompt text. Or a statement that no AI was used	Required, −10 if missing	TODO
`DESIGN-EXTENSION.md`	Answer to the expiry question: format changes, recovery changes, at least one subtle problem	Design extension	TODO
`diagrams/architecture.mmd`	Diagram source (Mermaid)	Open on screen during demo	TODO
`diagrams/architecture.png`	Rendered diagram	Open on screen during demo	TODO

How the parts connect



Files created at run time (not in the repository)

Path	Created by	Purpose
`kvdb`, `kvdb.exe`	Build command	CLI binary
`torture`, `torture.exe`	Build command	Torture tool binary
`<path>` (e.g. `db.kv`)	`open` or first write	The one authoritative data file
`<path>.lock`	`open`	Holds the OS lock while a process has the database open. Carries no data
`<path>.tmp`	Creation or compaction	Exists only during those operations. The next `open` deletes a leftover

Path	Created by	Purpose
`torture-work/db.kv`	Torture tool	Database under attack, shared across iterations
`torture-work/oracle.log`	Torture tool	Every ACKed operation
`torture-work/serve-stderr.log`	Torture tool	Stderr of each `serve` child
`torture-work/failure/`	Torture tool, on failure only	`db-as-killed.kv`, `db-after-recovery.kv`, `oracle.log`, `report.txt`

Mapping to the required layout (problem statement, section 10)

Required	This project	Reason
`README.md`, `ARCHITECTURE.md`, `BREAKING.md`, `PROMPTS.md`	Same names, repository root	As required
`diagrams/`	`diagrams/`	As required
`src/` (engine)	`src/kvdb/`	As required
`torture/` (tool)	`cmd/torture/`	A folder named `torture/` at the root would collide with the `./torture` binary on Linux and macOS
`tests/`	`tests/e2e_test.go`, plus unit tests beside the engine	Go convention
`.git/`	Created in your repository during the event	Must reflect work done during the event

Suggested build order

1. `format.go`: the bytes on disk. Check the encoding with a hex dump.
2. `write.go`: `Apply` and CP1 to CP7.
3. `recovery.go`: `Open`, `scan`, and CP8 to CP10 and CP14.
4. `compact.go` and `platform_*.go`: CP11 to CP13.
5. `check.go` and `db.go`: read-only check, `Get`, `Close`.
6. `cmd/kvdb`: CLI wrapper, then `serve`.
7. Unit tests in `kvdb_test.go`, run after each step.
8. `cmd/torture`: oracle first, then session, then modes.
9. Measure 1M-key recovery time and record the number.
10. Documents: README, ARCHITECTURE, BREAKING, DESIGN-EXTENSION, PROMPTS.